

Cookbook TDP - Grafi, DAO, Model e Ricorsione

Metodo pratico usato negli esercizi Chinook e facsimili d'esame

0. Regola generale

- Leggi la traccia e separa sempre: nodi, archi, peso, output, ricorsione.
- Il DAO deve preparare dati semplici: nodi, metriche, relazioni grezze.
- Il Model costruisce il grafo: dizionari, set, doppio ciclo sulle coppie, add_edge.
- Il Controller legge input, valida, chiama il Model e stampa.

Domanda da farti	Cosa produce
Chi sono i nodi?	Query dei nodi oppure lista oggetti caricati da id_map.
Quando esiste un arco?	Relazione grezza: nodo -> elemento condiviso oppure coppie dirette.
Come si calcola il peso?	COUNT, SUM, len(intersection), oppure somma di metriche.
Il grafo e orientato?	nx.DiGraph() se conta il verso; nx.Graph() se non conta.
Ci sono nodi isolati?	Query nodi separata; non costruire i nodi partendo solo dagli archi.

1. Frase della traccia -> schema da usare

Frase della traccia	Schema nostro
Due nodi sono collegati se hanno X in comune	DAO: (nodo_id, X). Model: dict nodo -> set(X), poi intersezione.
Due nodi sono collegati se compaiono nello stesso evento	DAO: (nodo_id, evento_id). Model: set(eventi) per ogni nodo.
Arco dal piu popolare al meno popolare	DAO: metrica per nodo. Model: confronto coppie e verso.
In parita inserire entrambi gli archi	Se v1 == v2: add_edge(n1,n2) e add_edge(n2,n1).
Peso = numero elementi comuni	peso = len(insieme1.intersection(insieme2)).
Peso = somma metriche dei due nodi	peso = valore1 + valore2.
Nodi isolati se non soddisfano la condizione	Aggiungi prima tutti i nodi, poi crea archi solo se condizione vera.
Stesso mese / anno	SQL: YEAR(data), MONTH(data); nel Model confronta (cliente, mese).
Estremi inclusi	SQL: BETWEEN inizio AND fine.
Top 5	Ordina in Python con reverse=True e ritorna result[:5].

2. Pattern DAO

2.1 Nodi tramite id_map

```
@staticmethod
def getAllNodes(filtro, id_map):
    conn = DBConnect.get_connection()
    result = []
    cursor = conn.cursor(dictionary=True)

    query = """
    SELECT DISTINCT x.Id
    FROM tabella x, ...
    WHERE ... = %s
    """

    cursor.execute(query, (filtro,))

    for row in cursor:
        result.append(id_map[row["Id"]])

    cursor.close()
```

```
conn.close()
return result
```

2.2 Metrica per nodo: id -> valore

```
@staticmethod
def getMetriche(filtro):
    conn = DBConnect.get_connection()
    result = {}
    cursor = conn.cursor(dictionary=True)

    query = """
        SELECT nodo_id, COUNT(*) AS valore
        FROM ...
        WHERE ... = %s
        GROUP BY nodo_id
    """

    cursor.execute(query, (filtro,))

    for row in cursor:
        result[row["nodo_id"]] = row["valore"]

    cursor.close()
    conn.close()
    return result
```

2.3 Relazione grezza: nodo -> elemento condiviso

```
@staticmethod
def getRelazioni(filtro):
    conn = DBConnect.get_connection()
    result = []
    cursor = conn.cursor(dictionary=True)

    query = """
        SELECT DISTINCT nodo_id, elemento_id
        FROM ...
        WHERE ... = %s
    """

    cursor.execute(query, (filtro,))

    for row in cursor:
        result.append((row["nodo_id"], row["elemento_id"]))

    cursor.close()
    conn.close()
    return result
```

3. Pattern Model per costruire il grafo

3.1 Grafo non orientato: condivisione di elementi

```
def build_graph(self, filtro):
    self._graph.clear()
    self._graph = nx.Graph()

    nodes = DAO.getAllNodes(filtro, self._id_map)
```

```

self._graph.add_nodes_from(nodes)

dati = DAO.getRelazioni(filtro) # (nodo_id, elemento_id)

elementi_per_nodo = {}
for nodo_id, elemento_id in dati:
    if nodo_id not in elementi_per_nodo:
        elementi_per_nodo[nodo_id] = set()
    elementi_per_nodo[nodo_id].add(elemento_id)

for i in range(len(nodes)):
    for j in range(i + 1, len(nodes)):
        n1 = nodes[i]
        n2 = nodes[j]

        ins1 = elementi_per_nodo.get(n1.Id, set())
        ins2 = elementi_per_nodo.get(n2.Id, set())
        comuni = ins1.intersection(ins2)

        if len(comuni) > 0:
            peso = len(comuni)
            self._graph.add_edge(n1, n2, weight=peso)

return self._graph.number_of_nodes(), self._graph.number_of_edges()

```

3.2 Grafo orientato: verso deciso da una metrica

```

def build_graph(self, filtro):
    self._graph.clear()
    self._graph = nx.DiGraph()

    nodes = DAO.getAllNodes(filtro, self._id_map)
    self._graph.add_nodes_from(nodes)

    metriche = DAO.getMetriche(filtro) # id -> valore
    dati = DAO.getRelazioni(filtro)

    elementi_per_nodo = {}
    for nodo_id, elemento_id in dati:
        if nodo_id not in elementi_per_nodo:
            elementi_per_nodo[nodo_id] = set()
        elementi_per_nodo[nodo_id].add(elemento_id)

    for i in range(len(nodes)):
        for j in range(i + 1, len(nodes)):
            n1 = nodes[i]
            n2 = nodes[j]

            id1 = n1.Id
            id2 = n2.Id

            comuni = elementi_per_nodo.get(id1, set()).intersection(
                elementi_per_nodo.get(id2, set()))

            if len(comuni) > 0:
                v1 = metriche.get(id1, 0)
                v2 = metriche.get(id2, 0)
                peso = v1 + v2

```

```

if v1 > v2:
    self_graph.add_edge(n1, n2, weight=peso)
elif v2 > v1:
    self_graph.add_edge(n2, n1, weight=peso)
else:
    self_graph.add_edge(n1, n2, weight=peso)
    self_graph.add_edge(n2, n1, weight=peso)

```

4. Collegamenti Chinook da ricordare

Obiettivo	Join da usare
Cliente -> fatture -> righe	customer c, invoice i, invoiceline il c.CustomerId = i.CustomerId i.InvoiceId = il.InvoiceId
Riga fattura -> brano	il.TrackId = t.TrackId
Brano -> album -> artista	t.AlbumId = al.AlbumId al.ArtistId = a.ArtistId
Brano -> genere	t.GenreId = g.GenreId
Brano -> media type	t.MediaTypeId = mt.MediaTypeId
Brano -> playlist	track t, playlisttrack pt t.TrackId = pt.TrackId
Playlist comuni	Composer/Album/Track -> PlaylistId, poi set/intersezione.
Stesso mese di acquisto	MONTH(i.InvoiceDate) AS mese; salva (CustomerId, mese).

5. Funzioni SQL utili negli esercizi

Funzione / costrutto	Quando usarlo	Esempio
YEAR(data)	Filtro per anno	YEAR(i.InvoiceDate) = %s
MONTH(data)	Stesso mese o raggruppamento mensile	MONTH(i.InvoiceDate) AS mese
BETWEEN	Range con estremi inclusi	i.InvoiceDate BETWEEN %s AND %s
COUNT(*)	Conta righe dopo join	COUNT(*) AS n
COUNT(DISTINCT x)	Evita duplicati	COUNT(DISTINCT i.InvoiceId)
SUM(campo)	Fatturato o peso cumulato	SUM(il.UnitPrice * il.Quantity)
GROUP BY	Aggregazione per nodo	GROUP BY a.ArtistId
HAVING	Filtro dopo aggregazione	HAVING COUNT(*) > 1
ORDER BY ... DESC	Ordinare top	ORDER BY valore DESC
LIMIT 5	Top 5 direttamente in SQL	ORDER BY valore DESC LIMIT 5
IS NOT NULL	Escludere NULL	t.Composer IS NOT NULL
!= "	Escludere stringhe vuote	t.Composer != "
LIKE	Filtro testuale parziale	Name LIKE '%Rock%'

6. Metodi di dettaglio nel Model

6.1 Nodo piu influente nei grafi orientati

```

def get_top_5_influenti(self):
    result = []

    for nodo in self_graph.nodes:
        peso_uscenti = 0
        peso_entranti = 0

        for vicino in self_graph.successors(nodo):
            peso_uscenti += self_graph[nodo][vicino]["weight"]

        for vicino in self_graph.predecessors(nodo):
            peso_entranti += self_graph[vicino][nodo]["weight"]

```

```

influenza = peso_uscenti - peso_entranti
result.append((nodo, influenza))

result.sort(key=lambda x: x[1], reverse=True)
return result[:5]

```

6.2 Componente connessa maggiore

```

def get_componente_maggiore(self):
    componenti = list(nx.connected_components(self._graph))
    if len(componenti) == 0:
        return []

    comp_max = max(componenti, key=len)
    result = list(comp_max)
    result.sort(key=lambda x: str(x))
    return result

```

6.3 Somma pesi incidenti in grafo non orientato

```

def get_somma_pesi_incidenti(self, nodo):
    somma = 0
    for vicino in self._graph.neighbors(nodo):
        somma += self._graph[nodo][vicino]["weight"]
    return somma

```

7. Ricorsione: schemi pronti

7.1 Cammino orientato da Start a End con K archi

```

def get_cammino_ottimo_k_archi(self, start, end, k):
    self._best_path = []
    self._best_peso = -1

    parziale = [start]
    self._ricorsione_k(parziale, end, k, 0)

    return self._best_path, self._best_peso

def _ricorsione_k(self, parziale, end, k, peso_parziale):
    ultimo = parziale[-1]
    archi_usati = len(parziale) - 1

    if archi_usati == k:
        if ultimo == end and peso_parziale > self._best_peso:
            self._best_peso = peso_parziale
            self._best_path = list(parziale)
        return

    for vicino in self._graph.successors(ultimo):
        if vicino not in parziale:
            peso_arco = self._graph[ultimo][vicino]["weight"]
            parziale.append(vicino)
            self._ricorsione_k(parziale, end, k, peso_parziale + peso_arco)
            parziale.pop()

```

7.2 Cammino con pesi strettamente crescenti

```
def get_cammino_peso_massimo_crescente(self, start):
    self._best_path = [start]
    self._best_peso = 0

    parziale = [start]
    self._ricorsione_crescente(parziale, None, 0)

    return self._best_path, self._best_peso

def _ricorsione_crescente(self, parziale, ultimo_peso, peso_parziale):
    ultimo = parziale[-1]

    if peso_parziale > self._best_peso:
        self._best_peso = peso_parziale
        self._best_path = list(parziale)

    for vicino in self._graph.neighbors(ultimo):
        if vicino not in parziale:
            peso_arco = self._graph[ultimo][vicino]["weight"]

            if ultimo_peso is None or peso_arco > ultimo_peso:
                parziale.append(vicino)
                self._ricorsione_crescente(parziale, peso_arco, peso_parziale + peso_arco)
            parziale.pop()
```

8. Controller: schemi fissi

8.1 Recuperare oggetto da dropdown

```
def get_selected_item(self, dropdown):
    for option in dropdown.options:
        if str(option.key) == str(dropdown.value):
            return option.data
    return None
```

8.2 Crea grafo

```
def handleCreaGrafo(self, e):
    self._view.txt_result.controls.clear()

    scelta = self.get_selected_item(self._view._ddScelta)
    if scelta is None:
        self._view.create_alert("Seleziona un valore.")
        return

    n, m = self._model.build_graph(scelta.Id)

    self._view.txt_result.controls.append(ft.Text("Grafo creato correttamente."))
    self._view.txt_result.controls.append(ft.Text(f"Numero nodi: {n}"))
    self._view.txt_result.controls.append(ft.Text(f"Numero archi: {m}"))
    self._view.update_page()
```

9. Errori tipici e correzione veloce

Errore	Perche succede	Correzione
cursor.execute(query, (x))	Non e una tupla	Usa (x,)
Oggetto vs stringa da dropdown	Hai usato dropdown.value direttamente	Usa get_selected_item
Nodi mancanti / isolati spariti	Hai creato nodi solo dagli archi	Query nodi separata
COUNT(*) troppo alto	Join produce duplicati	Usa COUNT(DISTINCT elemento)
A-B e B-A duplicati in grafo non orientato	Manca vincolo sulle coppie	Usa id1 < id2 oppure doppio for i,j
predecessors/successors non funzionano	Il grafo e nx.Graph()	Usa nx.DiGraph() oppure neighbors()
KeyError su dizionario	Nodo senza relazioni	Usa diz.get(id, set()) o diz.get(id, 0)
GROUP BY incompleto	Selezioni colonne non aggregate	Raggruppa tutte le colonne non aggregate
Date senza apici	SQL interpreta male	Usa parametri %s
NULL confrontato con !=	NULL non si confronta cosi	Usa IS NOT NULL

10. Procedura rapida da esame

- Scrivi in alto: NODI - ARCHI - PESO - OUTPUT - RICORSIONE.
- Crea prima le dataclass necessarie con __hash__ e __str__.
- Nel DAO scrivi query piccole: nodi, metriche, relazioni.
- Nel Model costruisci dizionari e set: sono la parte piu importante.
- Aggiungi sempre prima tutti i nodi al grafo.
- Crea archi con doppio ciclo sulle coppie.
- Scrivi metodi di dettaglio: nodi/archi, top 5, componente, influenti.
- Solo alla fine fai Controller e ricorsione.

Nota: questa cookbook usa volutamente query semplici e logica nel Model, per ridurre gli errori durante l'esame.